

# Projeto e Análise de Algoritmos

Árvore geradora mínima

Atílio G. Luiz

Primeiro Semestre de 2026

## Árvore geradora mínima

# Árvore geradora — Definição

## Definição

Seja  $G$  um grafo simples. Uma **árvore geradora** de  $G$  é um subgrafo gerador de  $G$  que é também uma árvore.

# Árvore geradora

## Fato 1

Todo grafo conexo contém uma árvore geradora.

Demonstração:

- ▶ segue facilmente do próximo resultado
- ▶ demonstre-o como exercício

## Lema

Seja  $G$  um grafo conexo e seja  $e$  uma aresta de  $G$ . Se  $e$  pertence a algum ciclo simples  $C$  de  $G$ , então  $G - e$  é conexo.

A recíproca também vale.

## Lema

Seja  $G$  um grafo conexo e seja  $e$  uma aresta de  $G$ . Se  $G - e$  é conexo então  $e$  pertence a algum ciclo simples de  $G$ .

# Introdução

Considere a seguinte situação:

- ▶ temos um conjunto de computadores
- ▶ para conectar dois computadores usamos um cabo
- ▶ sabemos o custo de conectar quaisquer dois computadores
- ▶ se  $A$  está conectado diretamente a  $B$ , e  $B$  está conectado diretamente a  $C$ , então  $A$  está conectado indiretamente a  $C$
- ▶ queremos que todos eles estejam interconectados

Como **minimizar** a quantidade de conexões e o custo total de cabo utilizado?

- ▶ este é um problema de otimização
- ▶ a entrada pode ser modelada como um grafo

# Árvore geradora mínima

Problema da árvore geradora mínima (AGM)

## Entrada:

- ▶ grafo conexo  $G = (V, E)$
- ▶ peso  $w(u, v) \geq 0$  para cada aresta  $(u, v)$

## Solução:

- ▶ subgrafo gerador conexo  $T$  de  $G$

## Objetivo:

- ▶ **minimizar**  $w(T) = \sum_{(u,v) \in T} w(u, v)$

# Refletindo um pouco

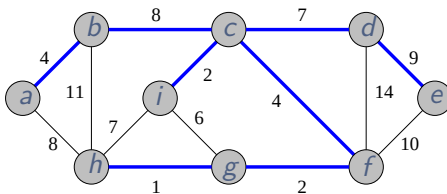
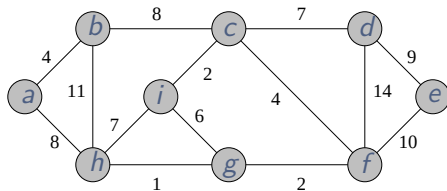
Observações:

- ▶ se o grafo fosse desconexo, então não haveria solução
- ▶ supomos que **não** há arestas de peso negativo.

Algumas perguntas:

- ▶ por que dizemos que uma solução ótima é uma **árvore**?
- ▶ e se houvesse arestas de peso negativo na entrada?

# Exemplo





Existem dois algoritmos **gulosos** que resolvem esse problema

1. algoritmo de Prim
2. algoritmo de Kruskal

# Esquema dos algoritmos

Ideia:

- ▶ iremos construir uma árvore incrementalmente
- ▶ denotamos por  $A$  o conjunto de arestas da árvore
- ▶ garantimos que  $A$  é um subconjunto de uma AGM
- ▶ mantemos essa invariante em cada iteração
  1. no início da iteração,  $A$  satisfaz a invariante
  2. selecionamos  $(u, v)$  tal que  $A \cup \{(u, v)\}$  também satisfaça
  3. adicionamos  $(u, v)$  ao conjunto  $A$

Dizemos que uma tal  $(u, v)$  é uma **aresta segura**.

# Algoritmo genérico

## AGM-Genérico( $G, w$ )

```
1   $A = \emptyset$ 
2  enquanto  $A$  não é uma árvore geradora
3      encontre uma aresta segura  $(u, v)$  para  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  devolva  $A$ 
```

Esse “algoritmo” está correto:

- ▶ o algoritmo devolve uma árvore geradora
- ▶ essa árvore é subgrafo de alguma AGM
- ▶ então ela também é mínima

Mas o algoritmo está bem definido?

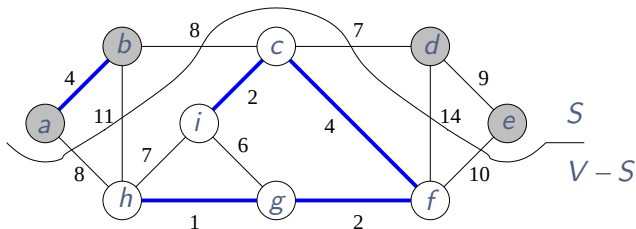
- ▶ se a iteração executa, então  $A$  não é árvore geradora
- ▶ então  $A$  não contém todas arestas de alguma AGM  $T$
- ▶ assim qualquer aresta de  $E[T] \setminus A$  é segura

Os algoritmos reais diferem em como encontrar uma aresta segura

# Como encontrar arestas seguras

Considere um grafo  $G = (V, E)$  e seja  $S \subset V$ .

- ▶ denote por  $\delta(S)$  o conjunto de arestas de  $G$  com um extremo em  $S$  e outro em  $V \setminus S$
- ▶ o conjunto  $\delta(S)$  é chamado de **corte**



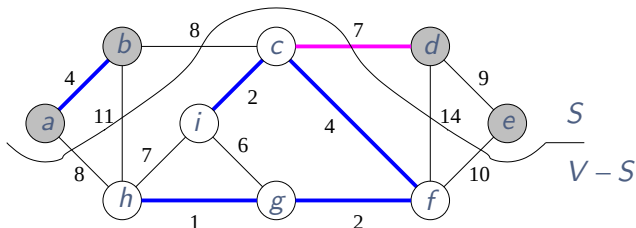
Um corte  $\delta(S)$  **respeita** um conjunto  $A$  de arestas se não contiver nenhuma aresta de  $A$ .

# Arestas leves

Uma aresta de um corte  $\delta(S)$  é **leve** se tem o menor peso entre as arestas do corte.

## Teorema

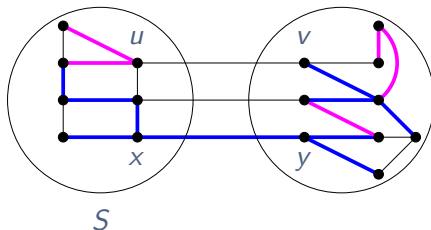
Seja  $(G, w)$  um grafo com pesos nas arestas e suponha que  $A$  é um subconjunto de arestas de uma AGM de  $G$ . Se  $\delta(S)$  é um corte que respeita  $A$  e  $(u, v)$  é uma aresta leve desse corte, então  $(u, v)$  é uma **aresta segura**.



# Prova do teorema

Seja  $T$  uma AGM que contém  $A$

- ▶ tome  $\delta(S)$  um corte que respeita  $A$
- ▶ e seja  $(u, v)$  uma **aresta leve** deste corte
- ▶ se  $(u, v)$  estiver em  $T$ , então não há nada a mostrar



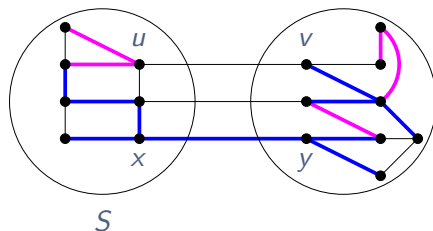
Suponha que  $(u, v)$  **não** é uma aresta de  $T$

- ▶ construiremos uma outra AGM  $T'$  que contém  $A \cup \{(u, v)\}$
- ▶ e daí concluiremos que  $(u, v)$  é **segura**

# Prova do teorema (cont)

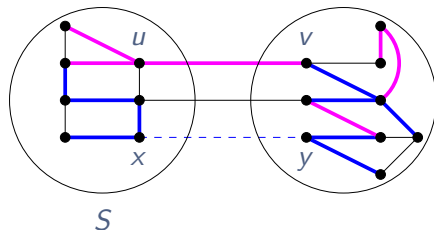
Existe um único caminho  $P$  de  $u$  a  $v$  em  $T$

- ▶  $u$  e  $v$  estão em lados opostos do corte  $\delta(S)$
- ▶ então alguma aresta de  $P$  pertence ao corte
- ▶ seja  $(x, y)$  uma tal aresta



Note que  $(x, y)$  não pertence a  $A$  pois o corte respeita  $A$

## Prova do teorema (cont)

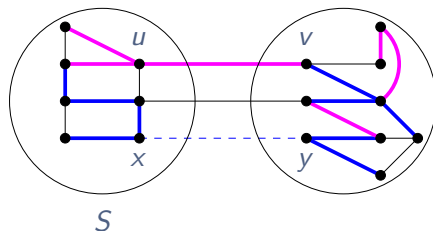


Defina  $T' := T - \{(x, y)\} \cup \{(u, v)\}$

- ▶ observe que  $T'$  é uma árvore geradora
- ▶ mostraremos que  $T'$  também é uma **AGM**



# Prova do teorema (cont)



Como  $(u, v)$  é uma **aresta leve** do corte  $\delta(S)$

- ▶ temos que  $w(u, v) \leq w(x, y)$  pois  $(x, y)$  pertence ao corte
- ▶ assim,  $w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$
- ▶ ademais, como  $w(T) \leq w(T')$ , temos que  $w(T) = w(T')$  e  $T'$  também é uma AGM

Além disso,  $T'$  contém  $A \cup \{(u, v)\}$

- ▶ portanto,  $(u, v)$  é uma **aresta segura**
- ▶ e concluimos a prova

# Consequência para os algoritmos

## Corolário

Seja  $(G, w)$  um grafo com pesos nas arestas e suponha que  $A$  é um subconjunto de arestas de uma AGM de  $G$ . Seja  $C$  o conjunto dos vértices de uma componente (árvore) da floresta  $G_A = (V, A)$ . Se  $(u, v)$  é uma aresta leve conectando  $C$  a alguma outra componente de  $G_A$ , então  $(u, v)$  é uma **aresta segura** para  $A$ .

Isso sugere um algoritmo iterativo

- ▶ Robert Prim e Joseph Kruskal implementaram essa ideia
- ▶ seus algoritmos fazem uso desse corolário

# Consequência para os algoritmos

- ▶ O **critério guloso** aqui é o próprio peso das arestas
- ▶ e a **escolha gulosa** consiste em escolher a aresta mais leve num corte que respeita  $A$ .
- ▶ Acabamos de provar que a escolha gulosa é **ótima**.

## Algoritmo de Prim

# O algoritmo de Prim

## Ideia

- ▶ no início, escolhemos um vértice  $r$  arbitrariamente
- ▶ o conjunto  $A$  são as arestas de uma árvore com raiz  $r$
- ▶ o conjunto  $S$  são os vértices dessa árvore
- ▶ em cada iteração, adicionamos uma **aresta leve** de  $\delta(S)$

## Detalhe de implementação importante

- ▶ como encontrar essa aresta leve **eficientemente**?

# Estruturas de dados

Como representar os vértices a serem adicionados?

- ▶ mantemos uma fila de prioridade (de mínimo)  $Q$
- ▶ ela contém todos vértices que **não** estão na árvore
- ▶ cada vértice  $v$  na fila tem prioridade  $key[v]$

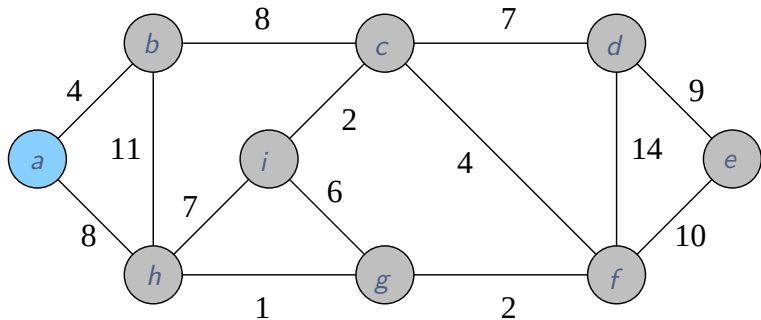
Qual a prioridade de escolher um vértice  $v$ ?

- ▶  $key[v]$  guarda o peso da menor aresta ligando  $v$  à árvore
- ▶ ou vale  $\infty$  se não houver uma tal aresta

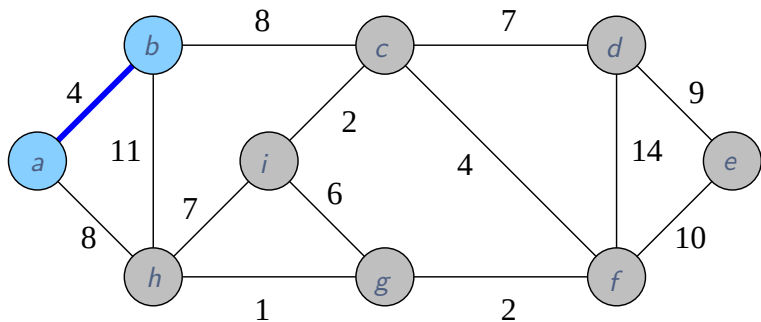
Como representar a árvore sendo construída?

- ▶ mantemos um vetor  $\pi$  de pais de todos vértices
- ▶ os vértices da árvore são  $S = V \setminus Q$
- ▶ e as arestas da árvore são  $A = \{(u, \pi[u]) : u \in S \setminus \{r\}\}$

# O algoritmo de Prim

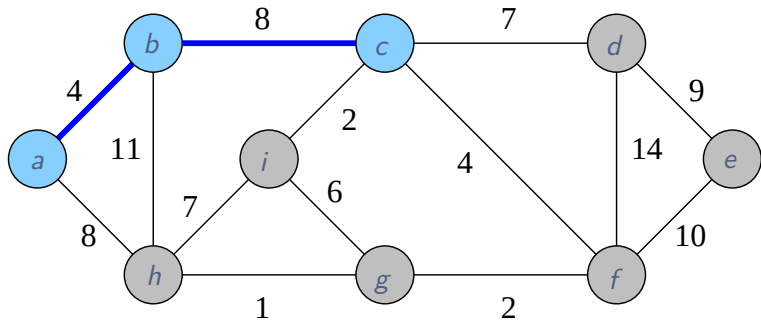


# O algoritmo de Prim

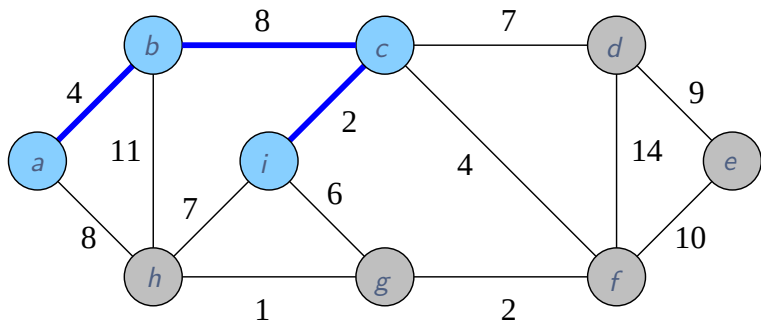




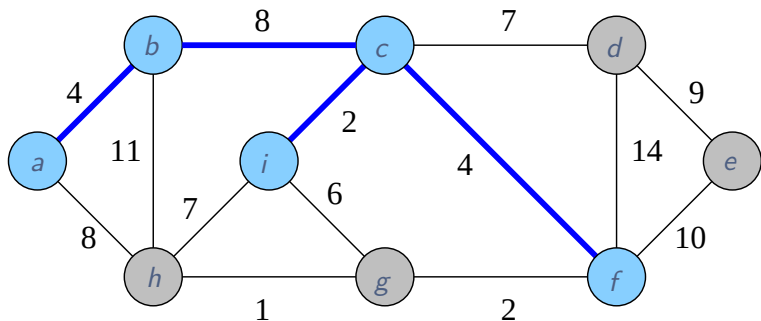
# O algoritmo de Prim



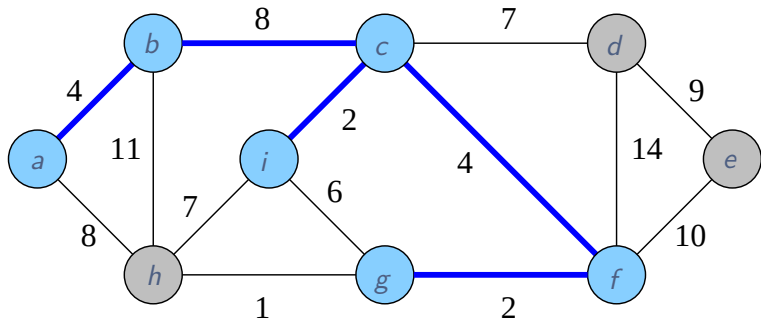
# O algoritmo de Prim



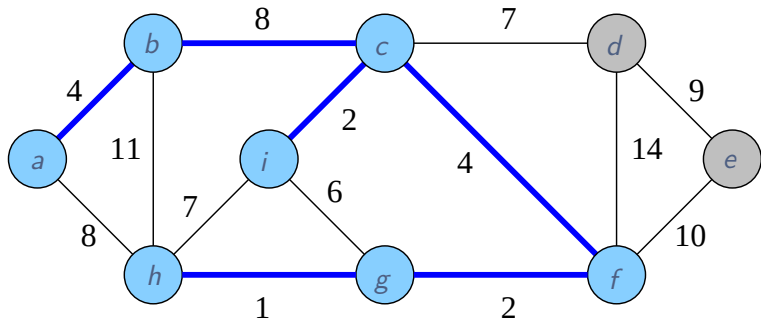
# O algoritmo de Prim



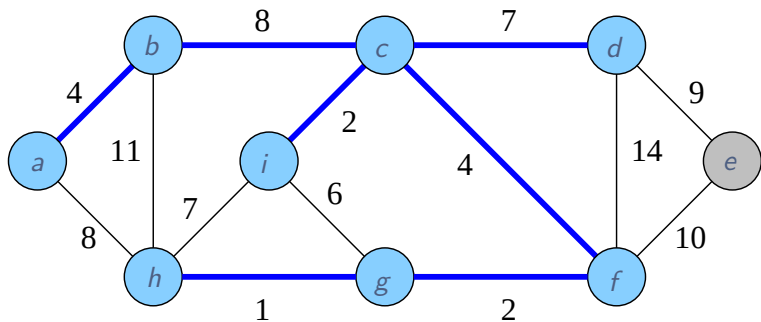
# O algoritmo de Prim



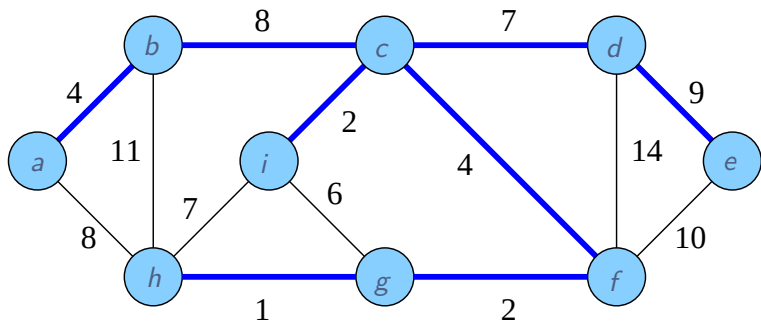
# O algoritmo de Prim



# O algoritmo de Prim



## O algoritmo de Prim



# O algoritmo de Prim

**AGM-Prim**( $G, w, r$ )

```
1  para cada  $u \in V(G)$  faça
2       $key[u] = \infty$ 
3       $\pi[u] = \text{NIL}$ 
4   $key[r] = 0$ 
5  cria fila de prioridade  $Q = V(G)$ 
6  enquanto  $Q \neq \emptyset$  faça
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      para cada  $v \in \text{Adj}[u]$ 
9          se  $v \in Q$  e  $w(u, v) < key[v]$ 
10             então  $\pi[v] = u$ 
11              $key[v] = w(u, v)$ 
```



# Correção do algoritmo

## Invariantes

Considere a execução no início do laço **enquanto** e defina  $S = V \setminus Q$  e  $A = \{(u, \pi[u]) : u \in S \setminus \{r\}\}$ , então:

1.  $A$  são arestas de uma árvore  $T$  com vértices  $S$  e raiz  $r$
  2. para cada  $v \in Q$ 
    - ▶ se  $\pi[v] \neq \text{NIL}$ , então  $\text{key}[v]$  é o peso de uma aresta com menor peso ligando  $v$  a algum vértice de  $T$
    - ▶ se  $\pi[v] = \text{NIL}$ , então não existe aresta ligando  $v$  a algum vértice de  $T$
- 
- ▶ as invariantes implicam que no início da iteração do laço,  $(u, \pi[u])$  é uma **aresta segura**
  - ▶ portanto, o algoritmo está correto

# Complexidade do algoritmo

A complexidade depende da fila de prioridade  $Q$

- ▶ cada teste  $v \in Q$  (linha 9) pode ser implementado para executar em tempo constante (como?)
- ▶ vamos contar quantas vezes executamos cada operação
  - ▶ Build-Min-Heap é executado 1 vez (linha 5)
  - ▶ EXTRACT-MIN é executada  $O(V)$  vezes (linha 6)
  - ▶ DECREASE-KEY é executada até  $O(E)$  vezes (linha 11)

Portanto o **tempo total** de execução é

Build-Min-Heap +  $O(V) \cdot \text{EXTRACT-MIN}$  +  $O(E) \cdot \text{DECREASE-KEY}$

# Complexidade usando min-heap

Se implementarmos  $Q$  como um min-heap, então

- ▶ Build-Min-Heap consome tempo  $O(V)$
- ▶ EXTRACT-MIN consome tempo  $O(\lg V)$
- ▶ DECREASE-KEY consome tempo  $O(\lg V)$

Então o tempo total será

$$O(V + V \lg V + E \lg V) = O(E \lg V)$$

## O algoritmo de Kruskal

# O algoritmo de Kruskal

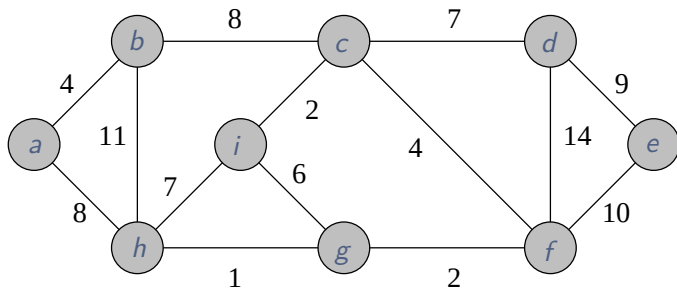
## Ideia

- ▶ o subgrafo  $G_A = (V, A)$  é uma floresta
- ▶ consideramos cada uma das arestas em ordem de peso
- ▶ em cada iteração, adicionamos uma aresta  $(u, v)$  se ela ligar duas componentes distintas  $C, C'$  da floresta
- ▶ note que  $(u, v)$  é uma **aresta leve** do  $\delta(C)$

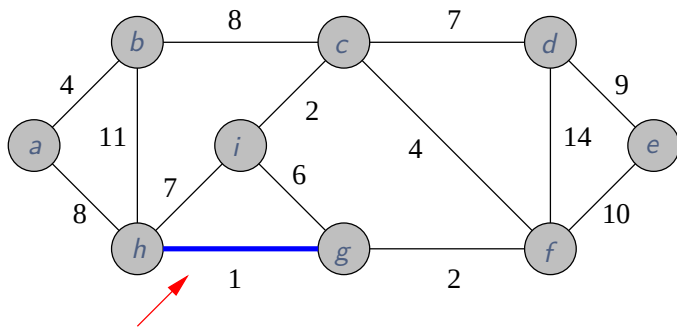
## Detalhe de implementação importante

- ▶ como saber se  $(u, v)$  liga componentes distintas?

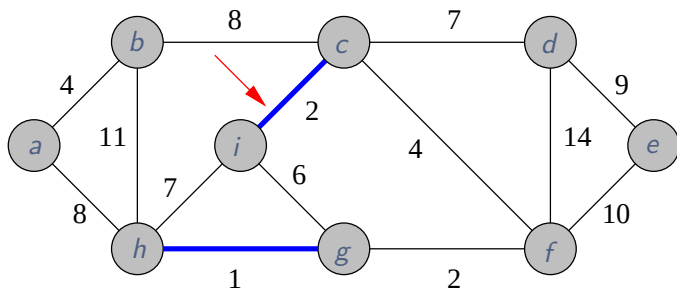
## O algoritmo de Kruskal



# O algoritmo de Kruskal

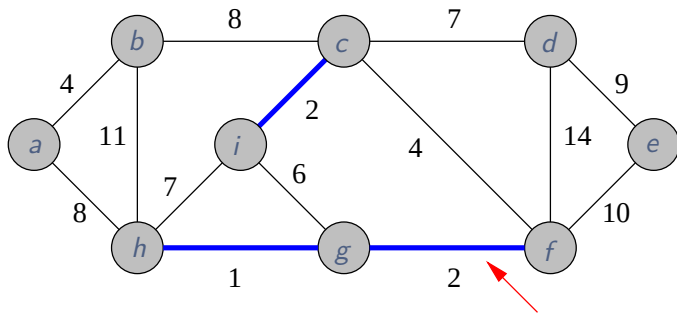


# O algoritmo de Kruskal

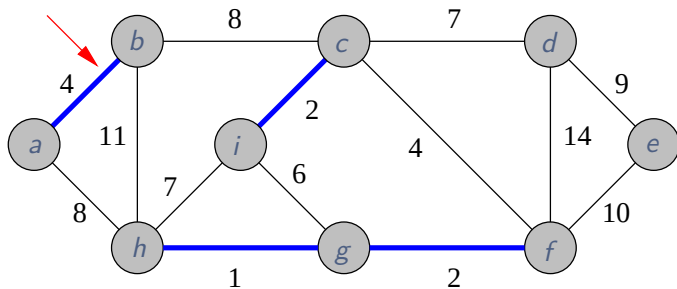




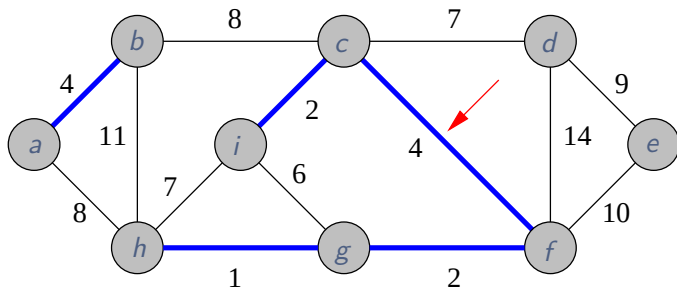
# O algoritmo de Kruskal



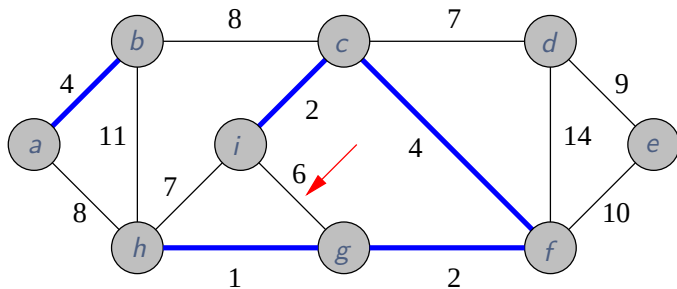
# O algoritmo de Kruskal



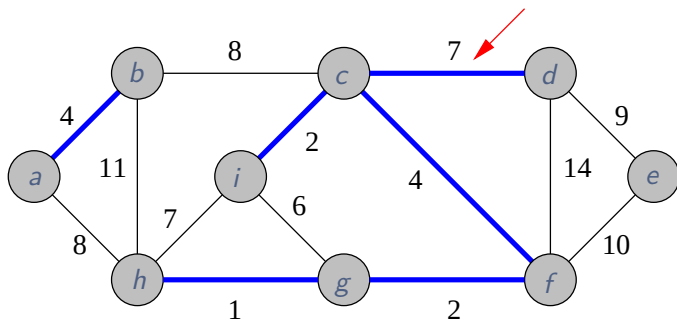
# O algoritmo de Kruskal



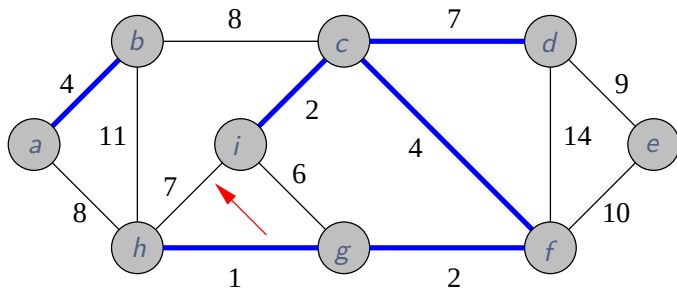
## O algoritmo de Kruskal



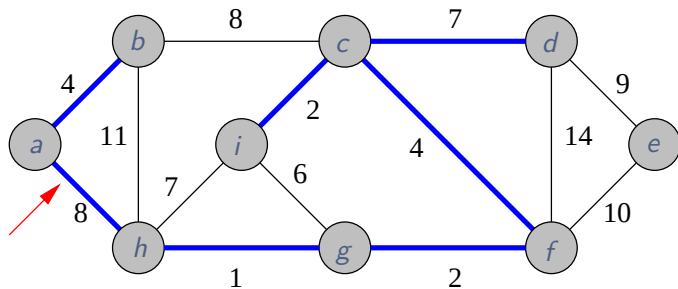
## O algoritmo de Kruskal



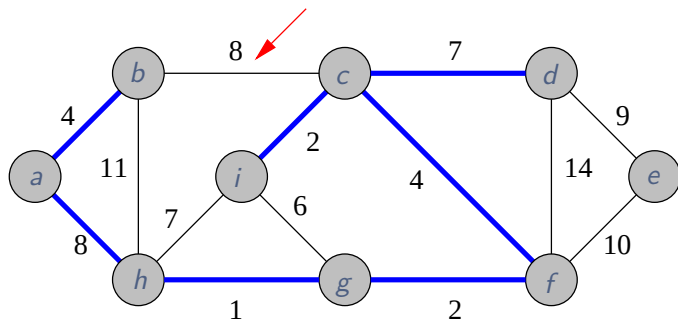
# O algoritmo de Kruskal



# O algoritmo de Kruskal

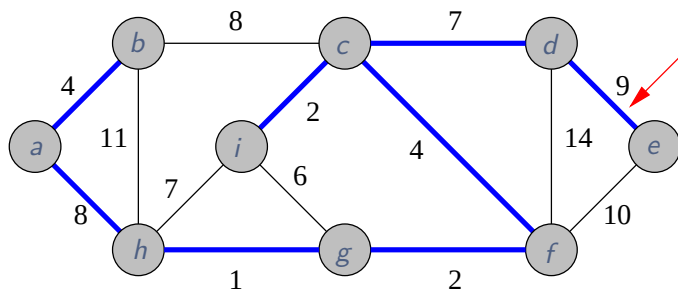


# O algoritmo de Kruskal

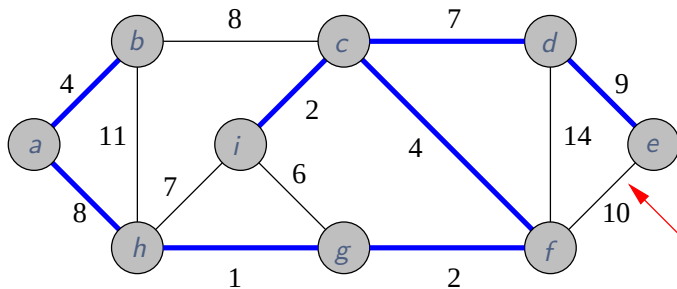




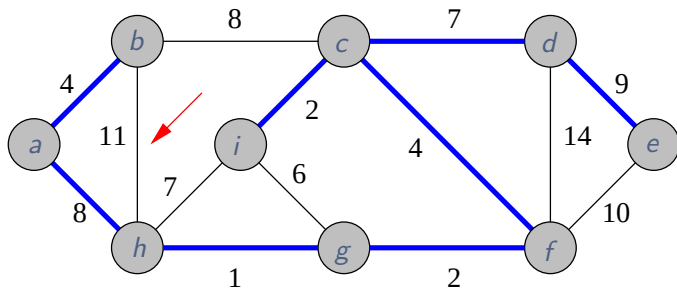
## O algoritmo de Kruskal



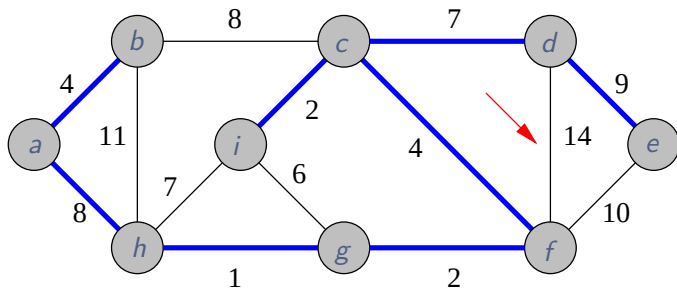
# O algoritmo de Kruskal



# O algoritmo de Kruskal



# O algoritmo de Kruskal



# O algoritmo de Kruskal

## AGM-Kruskal( $G, w$ )

```
1   $A = \emptyset$ 
2  ordene as arestas em ordem não decrescente de peso
3  para cada  $(u, v) \in E$  nessa ordem faça
4      se  $u$  e  $v$  estão em componentes distintos de  $(V, A)$ 
5          então  $A = A \cup \{(u, v)\}$ 
6  devolva  $A$ 
```

Esta é uma versão preliminar do algoritmo

- ▶ ainda falta detalhar como implementar a linha 4
- ▶ como fazer isso **eficientemente**?

# Estrutura de dados

Como guardar a floresta sendo construída?

- ▶ basta guardar o conjunto  $A$  das arestas
- ▶ assim a floresta é  $G_A = (V, A)$

Mas precisamos representar as componentes de  $G_A$

- ▶ durante o algoritmo, as componentes de  $G_A$  mudam
- ▶ queremos **determinar** qual componente contém vértice  $u$
- ▶ além de **fazer a união** das componentes que contêm  $u$  e  $v$

Qual estrutura realiza essas operações eficientemente?

# Conjuntos disjuntos

Queremos uma estrutura de dados

- ▶ que mantém coleção  $S_1, S_2, \dots, S_k$  de conjuntos disjuntos
- ▶ permite remover ou adicionar conjuntos à tal coleção
- ▶ e identifica cada conjunto por um representante
  - ▶ que é um elemento do próprio conjunto
  - ▶ a escolha do representante é irrelevante
  - ▶ mas o representante de um conjunto não pode mudar enquanto o conjunto não mudar

# Conjuntos disjuntos

A estrutura de dados deve permitir as seguintes operações:

1. **MAKE-SET( $x$ )**: cria um novo conjunto  $\{x\}$
2. **UNION( $x, y$ )**: une os conjuntos que contêm  $x$  e  $y$ 
  - ▶ se esses conjuntos forem  $S_x$  e  $S_y$
  - ▶ então adicionamos o conjunto  $S_x \cup S_y$
  - ▶ e descartamos  $S_x$  e  $S_y$  da coleção
3. **FIND-SET( $x$ )**: devolve o representante do conjunto com  $x$



# O algoritmo de Kruskal

Agora escrevemos a versão completa do algoritmo de Kruskal

**AGM-Kruskal**( $G, w$ )

```
1   $A = \emptyset$ 
2  para cada  $v \in V[G]$  faça
3      MAKE-SET( $v$ )
4  ordene as arestas em ordem não decrescente de peso
5  para cada  $(u, v) \in E$  nessa ordem faça
6      se FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          então  $A = A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  devolva  $A$ 
```

# Complexidade do algoritmo

A complexidade depende da estrutura de dados

- ▶ a ordenação toma tempo  $O(E \lg E)$
- ▶  $|V|$  chamadas a `MAKE-SET`
- ▶  $2|E|$  chamadas a `FIND-SET`
- ▶  $|V| - 1$  chamadas a `UNION`

Conjuntos disjuntos com florestas de conjuntos

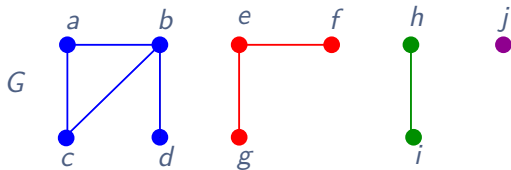
# Representação por florestas de conjuntos

- ▶ uma coleção de conjuntos é representada por uma floresta
- ▶ cada conjunto corresponde a uma árvore enraizada
- ▶ o representante de um conjunto é a raiz
- ▶ é a chamada **disjoint-set forest**

Veremos duas implementações

1. uma implementação simples
  - ▶ só altera a floresta durante **UNION**
  - ▶ o tempo não melhor que listas (assintoticamente)
2. uma implementação mais elaborada
  - ▶ utiliza heurísticas **union by rank** e **path compression**
  - ▶ também alteramos a floresta durante **FIND-SET**
  - ▶ é a melhor implementação conhecida

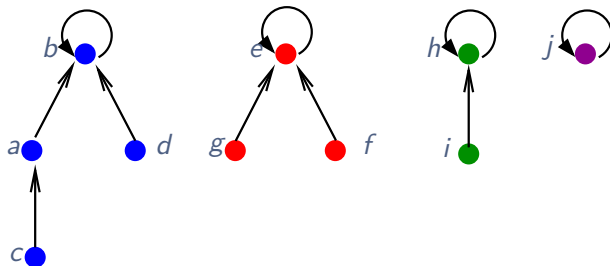
# Exemplo de grafo



Veja o grafo

- ▶ considere um conjunto para cada componente
- ▶ como representar os conjuntos com disjoint-set forest?

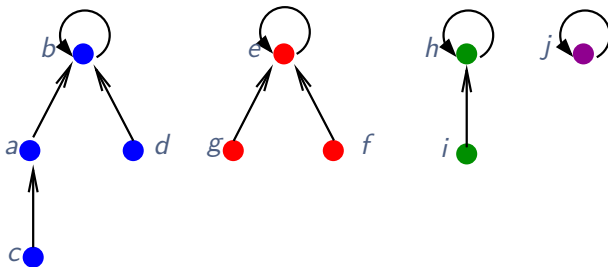
# Exemplo de representação



## Convenções

- ▶ cada conjunto é uma árvore enraizada
- ▶ cada elemento aponta para seu pai
- ▶ a raiz aponta para si mesma
- ▶ ela é o representante do conjunto

# Implementação simples



**Make-Set**( $x$ )

1  $pai[x] = x$

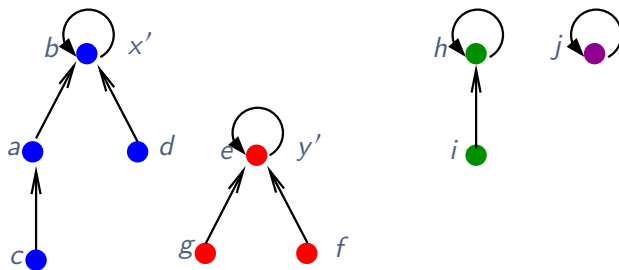
**Find-Set**( $x$ )

1 se  $x = pai[x]$

2 então devolva  $x$

3 senão devolva **FIND-SET**( $pai[x]$ )

# Implementação simples (cont)

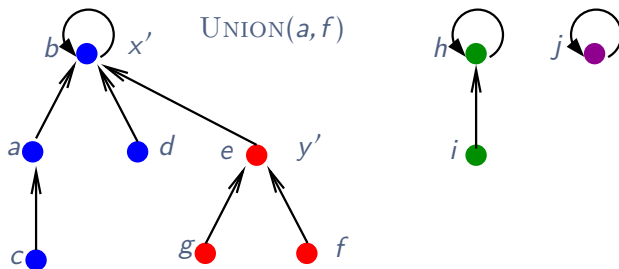


**Union**( $x, y$ )

- 1  $x' = \text{FIND-SET}(x)$
- 2  $y' = \text{FIND-SET}(y)$
- 3  $\text{pai}[y'] = x'$



# Implementação simples (cont)



## **Union**( $x, y$ )

- 1  $x' = \text{FIND-SET}(x)$
- 2  $y' = \text{FIND-SET}(y)$
- 3  $\text{pai}[y'] = x'$

# Complexidade da implementação simples

Tempo das operações

- ▶  $\text{MAKE-SET}(x) - O(1)$
- ▶  $\text{FIND-SET}(x) - O(n)$
- ▶  $\text{UNION}(x, y) - O(n)$

Não é ainda uma boa representação

- ▶ considere uma sequência de  $n - 1$  chamadas a  $\text{UNION}$
- ▶ que resulta em uma cadeia linear com  $n$  nós
- ▶ daí,  $n$  chamadas a  $\text{FIND-SET}$  pode levar tempo total  $\Theta(n^2)$

Podemos melhorar isso usando duas heurísticas

- ▶ union by rank
- ▶ path compression

# Union by rank

- ▶ cada nó  $x$  está associado a um número  $rank[x]$ 
  - ▶ pode ser a altura de  $x$  na árvore,
  - ▶ ou pode ser um número menor
- ▶ a raiz com menor  $rank$  aponta para a raiz com maior  $rank$

# Union by rank

## **Make-Set**( $x$ )

- 1  $pai[x] = x$
- 2  $rank[x] = 0$

## **Union**( $x, y$ )

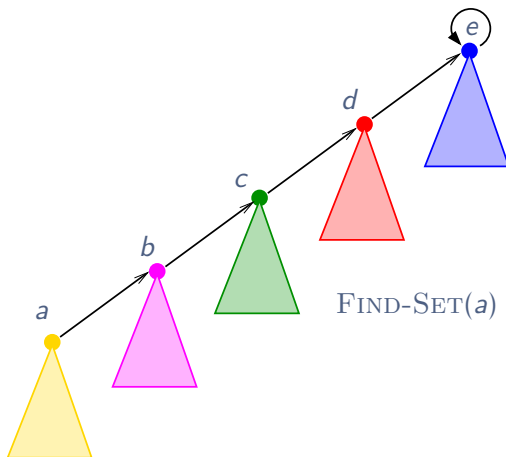
- 1  $LINK(FIND-SET(x), FIND-SET(y))$

## **Link**( $x, y$ )

- 1 se  $rank[x] > rank[y]$  então
- 2      $pai[y] = x$
- 3 senão
- 4      $pai[x] = y$
- 5     se  $rank[x] = rank[y]$  então
- 6          $rank[y] = rank[y] + 1$

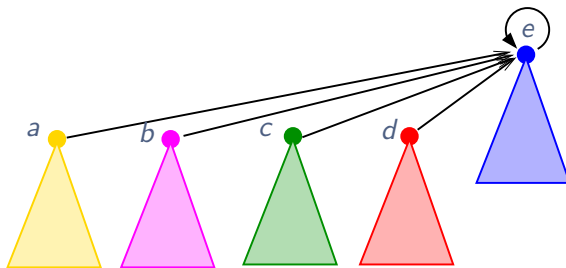
# Path compression

A ideia é muito simples: ao tentar determinar o representante (**raiz** da árvore) de um nó fazemos com que todos os nós no caminho apontem para a raiz.



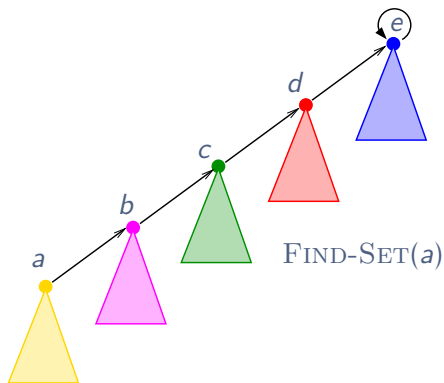
# Path compression

A ideia é muito simples: ao tentar determinar o representante (**raiz** da árvore) de um nó fazemos com que todos os nós no caminho apontem para a raiz.



$\text{FIND-SET}(a)$

# Path compression



**Find-Set**( $x$ )

- 1 se  $x \neq \text{pai}[x]$
- 2     então  $\text{pai}[x] = \text{FIND-SET}(\text{pai}[x])$
- 3     devolva  $\text{pai}[x]$

# Complexidade com as duas heurísticas

## Teorema (Tarjan)

Uma sequência de  $m$  operações MAKE-SET, UNION e FIND-SET pode ser executada com **disjoint-set forest** com union by rank e path compression em tempo  $O(m\alpha(n))$  no pior caso.

Não vamos demonstrar este teorema

- ▶ ele implica que o custo amortizado por chamada é  $\alpha(n)$
- ▶ o valor de  $\alpha(n)$  cresce arbitrariamente com  $n$
- ▶ mas num ritmo realmente devagar
- ▶ uma demonstração está em CLRS



# Estimando o tempo amortizado

Quão pequeno é o custo de cada operação?

$$\alpha(n) = \begin{cases} 0 & \text{para } 0 \leq n \leq 2, \\ 1 & \text{para } n = 3, \\ 2 & \text{para } 4 \leq n \leq 7, \\ 3 & \text{para } 8 \leq n \leq 2047, \\ 4 & \text{para } 2048 \leq n \leq 16^{512} \end{cases}$$

- ▶ observe que  $16^{512}$  é muito muito maior que  $10^{80}$ , o número estimado de átomos do universo!
- ▶ isso significa que na prática o custo amortizado de cada chamada é limitado pela **constante**, digamos 4
- ▶ vamos utilizar essa estrutura no algoritmo de Kruskal

# O algoritmo de Kruskal (de novo)

## AGM-Kruskal( $G, w$ )

```
1   $A = \emptyset$ 
2  para cada  $v \in V[G]$  faça
3      MAKE-SET( $v$ )
4  ordene as arestas em ordem não decrescente de peso
5  para cada  $(u, v) \in E$  nessa ordem faça
6      se FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7          então  $A = A \cup \{(u, v)\}$ 
8              UNION( $u, v$ )
9  devolva  $A$ 
```

Complexidade:

- ▶ ordenação das arestas: tempo  $O(E \lg E)$
- ▶  $O(E)$  chamadas a MAKE-SET, FIND-SET e UNION: tempo  $O(E\alpha(V))$
- ▶ o tempo total é dominado pelo tempo de ordenação
- ▶ mas as demais operações têm tempo praticamente linear!